Automation & QA Developer

Take-Home Skills Assessment - Complete Submission

Candidate: [Your Name] | **GitHub:** [repo link] | **Loom:** [video link]

This document contains every deliverable requested: Task 1 QA report, Task 2 n8n workflow (documentation and full exported JSON), the Bonus uptime monitor (documentation and full exported JSON), the walkthrough script, and the submission checklist.

Section	Content
1	Task 1 - QA & Debug Report (bug table + root-cause analysis)
2	Task 2 - n8n API Integration Workflow (README, node-by-node design, test steps)
3	Bonus - Uptime Monitor
4	Loom walkthrough script and interview Q&A prep
Appendix A	Task2_Workflow_YourName.json (full export)
Appendix B	Bonus_UptimeMonitor_YourName.json (full export)

1. Task 1 - QA & Debug Report

App: RealWorld "Conduit" (https://demo.realworld.io). **Areas reviewed:** sign-up, login, create / edit / delete article, comments, settings, log out; plus security, accessibility, validation and error-handling checks.

Summary for a non-technical stakeholder. The core flows work, but the app is not production-ready. The biggest concerns: (1) the login token is kept where any injected script can steal it, (2) destructive actions such as deleting an article happen instantly with no confirmation, and (3) forms give weak, unclear feedback and are hard to use with a screen reader. None are hard to fix; the table lists what to change and how urgent it is.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Login token stored in localStorage	1. Log in. 2. DevTools > Application > Local Storage. 3. Find the JWT.	Expected: token in HttpOnly Secure cookie, unreadable by JS. Actual: readable by any script; one XSS bug = account takeover.	High	Frontend uses localStorage.setItem and an Authorization header instead of cookie sessions.
2	Delete article has no confirmation	1. Log in, open your article. 2. Click Delete Article.	Expected: confirm dialog or undo. Actual: removed immediately and permanently.	Medium	Delete button wired directly to DELETE API call.
3	Inputs have placeholders but no labels	1. Open Sign up / Sign in / Settings. 2. Inspect or run Lighthouse accessibility audit.	Expected: label or aria-label per input. Actual: placeholder only; not reliably announced by screen readers.	Medium	Markup relies on placeholder as label (WCAG 1.3.1 / 3.3.2).
4	Weak validation, no password rules	1. Sign up with password "1", a 200+ char username, special characters, and an existing email.	Expected: clear inline rules and messages. Actual: weak passwords accepted; errors are raw server strings at the top of the form.	Medium	Minimal backend validation; frontend shows server errors verbatim.
5	No login rate limiting	1. Submit wrong credentials 20+ times per minute via UI or curl to /api/users/login.	Expected: throttle, lockout or CAPTCHA. Actual: each attempt processed equally.	Medium	No rate-limit middleware or lockout logic.
6	Poor handling of API failures	1. DevTools > Network > Offline. 2. Try to publish an article or load the feed.	Expected: clear retry message, draft kept. Actual: no clear message; typed content can be lost.	Medium	No global error interceptor; form state not persisted.
7	Rendered markdown may allow HTML injection	1. Create an article with <img src=x onerror=alert(1)> in the body. 2. View it.	Expected: content sanitised. Actual: depends on the markdown renderer's sanitiser; if not strict, script runs (chains with issue 1).	Low (High if it executes)	Client-side markdown-to-HTML without allow-list sanitiser (e.g. DOMPurify).
8	Long text breaks layout	1. Create an article with a 300+ char title and a long unbroken string in the body.	Expected: wrap or ellipsis. Actual: overflow in cards/feed, worst on mobile widths.	Low	No max-length limits or CSS word-break rules.

Root-Cause Analysis - Issue #1: JWT stored in localStorage

After login, the API returns a JSON Web Token and the frontend saves it with `localStorage.setItem`, then attaches it to every request in an Authorization header. Anything in localStorage is readable by any JavaScript on the page, including scripts injected through an XSS flaw, a compromised npm dependency or a browser extension. Because articles and comments accept user-written markdown, there is a realistic path for injected script to run in another user's session and read the token. A stolen token can be replayed from anywhere until it expires, and the app has no server-side way to revoke it. This happens because the design uses stateless bearer tokens with client-side storage, which is easy to build (especially with AI tools) but weak against script injection. **Fix:** issue the session in an HttpOnly, Secure, SameSite=Lax cookie set by the server; add CSRF protection; shorten token lifetime with refresh-token rotation; sanitise all rendered markdown with a strict sanitiser (e.g. DOMPurify) and add a Content-Security-Policy header. **Verify:** afterwards, document.cookie and localStorage must not expose the token, and an injected script payload must not execute.

2. Task 2 - n8n API Integration Workflow: GitHub "Morning Brief"

Flow: Schedule Trigger (every 1 hour) → HTTP Request: GitHub Search API → Code: keep top 5 by stars → HTTP Request: GitHub README endpoint (enrichment) → Code: build digest → IF stars > 1000 → Discord webhook (HOT message / normal message). Error branch: failed search → Code: format error → Discord alert.

Requirement	How it is met
Trigger	Schedule Trigger, interval 1 hour (for testing).
First HTTP request	GET api.github.com/search/repositories?q=topic:n8n&sort=stars&order=desc&per_page=10. Retries 3 times, 15s timeout.
Transformation	Code node maps to name, stars, URL, description, language; sorts by stars; keeps top 5; throws a clear error if zero results.
Second HTTP request	GET /repos/{owner}/{repo}/readme for the #1 repo with Accept: application/vnd.github.raw+json; a 250-character snippet goes in the digest.
Conditional branch	IF top repo stars > 1000: 'HOT' Discord message, otherwise normal message. 1000 = a sensible 'notable project' cut-off.
Output	Discord webhook (free, one-step setup).
Error handling	Search node: On Error = Continue (using error output) after 3 retries; error output formats a warning and posts it to Discord. README node: Continue, so digest still ships with 'README unavailable'. Discord nodes: Continue, so a failed send cannot crash the run.
Credentials	GitHub API credential and Discord Webhook credential from n8n's Credentials store; no secrets in the JSON (only placeholder credential IDs).

Why GitHub: free, no sign-up needed to test, stable JSON, and both required calls use one API and one credential. Authenticated calls raise the rate limit from 60 to 5,000 requests per hour.
How to import and test: n8n → Workflows → Import from File → select Appendix A JSON → attach your GitHub and Discord Webhook credentials → click Execute Workflow. To test the error path, change the search URL to `/search/repositoriez` and execute again; a warning should arrive in Discord.
Optional curl-style check: the workflow uses a Schedule trigger, so no webhook is needed; the manual Execute button is the test.

3. Bonus - Uptime Monitor

Target: the app from Task 1 (https://demo.realworld.io). **Notification channel:** same Discord webhook as Task 2.

- **Every 5 Minutes** trigger → **Start Timer** (Code, records start time and URL).
- **HTTP Health Check:** GET with 10s timeout, Never Error on, 3 retries with 3s waits, Continue on error.
- **Evaluate Result** (Code): reads HTTP status (or 0 on timeout/DNS failure), calculates response time in ms, and updates rolling stats (checks, failures, total and max latency) in workflow static data.
- **Is App Down?** (IF): if status is not 200 → **Discord: Down Alert** with status, response time and timestamp.
- **Daily 9AM** trigger → **Build Summary** (checks, failures, uptime %, average and slowest response) → Discord, then counters reset.
- Note: static data persists only while the workflow is active; test the daily summary by executing that branch manually.

4. Walkthrough Script (Loom, ~5-6 min) and Interview Prep

- **Intro (20s):** name and the three parts: QA report, n8n workflow, bonus monitor.
- **Task 1 (2 min):** scroll the bug table; demo issue 1 (DevTools → localStorage token) and issue 2 (delete without confirmation); explain root cause and fix for issue 1.
- **Task 2 (2 min):** walk the canvas left to right; explain why GitHub, top-5 in a Code node, README enrichment, 1000-star threshold; Execute and show the Discord message; break the URL to show the error alert; show the Credentials store.
- **Bonus (1 min):** point the URL to a bad host, run it, show the DOWN alert; mention retries, latency tracking, daily summary.
- **Close (20s):** next steps: Error Trigger workflow, metrics to Google Sheets, Playwright tests.

Likely interview questions

Question	Short answer
Why GitHub API?	Free, reliable, two endpoints under one credential, easy to demo.
Why a Code node rather than expressions?	Filter + map + sort + slice is clearer and easier to test in one place.
Why 1000 stars?	Separates notable projects from the rest; easy to change in the IF node.
What if GitHub is down or rate-limited?	Retries 3 times, then error output posts a Discord alert; the run never fails silently.
Why continue on fail for README?	Enrichment is nice-to-have; the digest should still be delivered.
Where are secrets?	n8n Credentials store; exported JSON contains only credential IDs.
Why is JWT in localStorage risky?	Any injected script can read it; use HttpOnly cookies, CSRF protection, CSP and sanitised markdown.
How would you improve the monitor?	Persist metrics to a sheet/DB, add an Error Trigger workflow, alert only after N consecutive failures.

Appendix A - Task2_Workflow_YourName.json


```
{
 "name": "Task2 - GitHub Morning Brief",
 "nodes": [
  {
   "parameters": {
    "rule": {
     "interval": [
      {
       "field": "hours",
       "hoursInterval": 1
      }
     ]
    }
   },
   "id": "002ad375-c8ba-425a-b87b-3751fbb90080",
   "name": "Every 1 Hour",
   "type": "n8n-nodes-base.scheduleTrigger",
   "typeVersion": 1.2,
   "position": [
    0,
    300
   ]
  },
  {
   "parameters": {
    "url": "https://api.github.com/search/repositories",
    "authentication": "predefinedCredentialType",
    "nodeCredentialType": "githubApi",
    "sendQuery": true,
    "queryParameters": {
     "parameters": [
      {
       "name": "q",
       "value": "topic:n8n"
      },
      {
       "name": "sort",
       "value": "stars"
      },
      {
       "name": "order",
       "value": "desc"
      },
      {
       "name": "per_page",
       "value": "10"
      }
     ]
    },
    "options": {
     "timeout": 15000
    }
   },
   "id": "6cb033d4-bacc-4181-a64d-1f446cab12bd",
   "name": "GitHub: Search Repos",
   "type": "n8n-nodes-base.httpRequest",
   "typeVersion": 4.2,
   "position": [
    240,
    300
   ],
   "credentials": {
    "githubApi": {
     "id": "REPLACE_GITHUB_CRED_ID",
     "name": "GitHub account"
    }
   },
   "onError": "continueErrorOutput",
   "retryOnFail": true,
   "maxTries": 3,
   "waitBetweenTries": 2000
  },
  {
   "parameters": {
    "jsCode": "// Reshape GitHub search response: keep only useful fields, sort by stars, keep top 5\nconst items = $input.first().json.items ||
[];\nconst top5 = items\n  .map(r => ({\n    name: r.full_name,\n    stars: r.stargazers_count,\n    url: r.html_url,\n    description: r.description
|| 'No description',\n    language: r.language || 'n/a'\n  }))\n  .sort((a, b) => b.stars - a.stars)\n  .slice(0, 5);\n\nif (top5.length === 0) {\n
throw new Error('GitHub returned 0 repositories');\n}\nreturn [{ json: { top5, best: top5[0] } }];"
   },
   "id": "e9e69231-d0e5-4488-a66f-f7c928c2ca5b",
   "name": "Top 5 Repos",
   "type": "n8n-nodes-base.code",
   "typeVersion": 2,
   "position": [
    480,
    200
   ]
  },
  {
   "parameters": {
    "url": "=https://api.github.com/repos/{{ $json.best.name }}/readme",
    "authentication": "predefinedCredentialType",
    "nodeCredentialType": "githubApi",
    "sendHeaders": true,
    "headerParameters": {
     "parameters": [
      {
       "name": "Accept",
       "value": "application/vnd.github.raw+json"
      }
```

```
    ]
   },
   "options": {
    "timeout": 15000,
    "response": {
     "response": {
      "responseFormat": "text"
     }
    }
   }
  },
  "id": "32055092-14db-4a11-b29e-8c5bd68a51ff",
  "name": "GitHub: Fetch README",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 4.2,
  "position": [
   720,
   200
  ],
  "credentials": {
   "githubApi": {
    "id": "REPLACE_GITHUB_CRED_ID",
    "name": "GitHub account"
   }
  },
  "onError": "continueRegularOutput"
 },
 {
  "parameters": {
   "jsCode": "// Build the digest text. README enrichment may have failed -> fall back gracefully.\nconst t = $('Top 5 Repos').first().json;\nconst
enr = $input.first().json;\nlet readme = (typeof enr.data === 'string') ? enr.data : '';\nconst snippet = readme\n  ? readme.replace(/[#*`>\\[\\]!]/g,
 '').replace(/\\s+/g, ' ').trim().slice(0, 250)\n  : 'README unavailable (enrichment call failed)';\n\nconst lines = t.top5.map((r, i) =>\n  `${i +
1}. **${r.name}** ■ ${r.stars} (${r.language})\\n   ${r.description}\\n   ${r.url}`);\n\nconst content =\n  `■ **Morning Brief – Top n8n repos on
GitHub**\\n\\n` +\n  lines.join('\\n\\n') +\n  `\\n\\n■ **Spotlight: ${t.best.name}**\\n${snippet}`;\n\nreturn [{ json: { content: content.slice(0,
1900), stars: t.best.stars, best: t.best.name } }];"
  },
  "id": "fb1dab9f-4168-4dff-83b6-0a187e2495b6",
  "name": "Build Digest",
  "type": "n8n-nodes-base.code",
  "typeVersion": 2,
  "position": [
   960,
   200
  ]
 },
 {
  "parameters": {
   "conditions": {
    "options": {
     "caseSensitive": true,
     "leftValue": "",
     "typeValidation": "strict"
    },
    "conditions": [
     {
      "id": "c1",
      "leftValue": "={{ $json.stars }}",
      "rightValue": 1000,
      "operator": {
       "type": "number",
       "operation": "gt"
      }
     }
    ],
    "combinator": "and"
   },
   "options": {}
  },
  "id": "0375e404-ef12-4971-922d-69ac9a55149d",
  "name": "Stars > 1000?",
  "type": "n8n-nodes-base.if",
  "typeVersion": 2,
  "position": [
   1200,
   200
  ]
 },
 {
  "parameters": {
   "authentication": "webhook",
   "content": "=■ **HOT: top repo has over 1000 stars!**\n\n{{ $json.content }}",
   "options": {}
  },
  "id": "17ccbd7a-e692-4abb-9888-43e694d68f57",
  "name": "Discord: Hot Digest",
  "type": "n8n-nodes-base.discord",
  "typeVersion": 2,
  "position": [
   1440,
   120
  ],
  "credentials": {
   "discordWebhookApi": {
    "id": "REPLACE_DISCORD_CRED_ID",
    "name": "Discord Webhook (Morning Brief)"
   }
  },
  "onError": "continueRegularOutput"
 },
 {
  "parameters": {
   "authentication": "webhook",
   "content": "={{ $json.content }}",
   "options": {}
  },
```

```
    "id": "13e06a23-3d28-47fe-b6c7-cb20cf5dd0f9",
    "name": "Discord: Normal Digest",
    "type": "n8n-nodes-base.discord",
    "typeVersion": 2,
    "position": [
     1440,
     300
    ],
    "credentials": {
     "discordWebhookApi": {
      "id": "REPLACE_DISCORD_CRED_ID",
      "name": "Discord Webhook (Morning Brief)"
     }
    },
    "onError": "continueRegularOutput"
   },
   {
    "parameters": {
     "jsCode": "// Fallback path: the GitHub call failed. Log + notify instead of crashing silently.\nconst err = $input.first().json;\nconst msg =
(err.error && (err.error.message || err.error.description)) || JSON.stringify(err).slice(0, 300);\nconsole.log('Morning Brief FAILED:', msg);\nreturn
[{ json: { content: `■■ **Morning Brief failed** at ${new Date().toISOString()}\\nGitHub API error: ${msg}` } }];"
    },
    "id": "79e1a8e2-4cf5-4415-9ae3-98a96ce2e7f5",
    "name": "Format Error",
    "type": "n8n-nodes-base.code",
    "typeVersion": 2,
    "position": [
     480,
     420
    ]
   },
   {
    "parameters": {
     "authentication": "webhook",
     "content": "={{ $json.content }}",
     "options": {}
    },
    "id": "b2ec2899-37b2-45c4-bc76-378cc52dab57",
    "name": "Discord: Error Alert",
    "type": "n8n-nodes-base.discord",
    "typeVersion": 2,
    "position": [
     720,
     420
    ],
    "credentials": {
     "discordWebhookApi": {
      "id": "REPLACE_DISCORD_CRED_ID",
      "name": "Discord Webhook (Morning Brief)"
     }
    },
    "onError": "continueRegularOutput"
   }
  ],
  "connections": {
   "Every 1 Hour": {
    "main": [
     [
      {
       "node": "GitHub: Search Repos",
       "type": "main",
       "index": 0
      }
     ]
    ]
   },
   "GitHub: Search Repos": {
    "main": [
     [
      {
       "node": "Top 5 Repos",
       "type": "main",
       "index": 0
      }
     ],
     [
      {
       "node": "Format Error",
       "type": "main",
       "index": 0
      }
     ]
    ]
   },
   "Top 5 Repos": {
    "main": [
     [
      {
       "node": "GitHub: Fetch README",
       "type": "main",
       "index": 0
      }
     ]
    ]
   },
   "GitHub: Fetch README": {
    "main": [
     [
      {
       "node": "Build Digest",
       "type": "main",
       "index": 0
      }
     ]
    ]
```

```
  },
  "Build Digest": {
   "main": [
    [
     {
      "node": "Stars > 1000?",
      "type": "main",
      "index": 0
     }
    ]
   ]
  },
  "Stars > 1000?": {
   "main": [
    [
     {
      "node": "Discord: Hot Digest",
      "type": "main",
      "index": 0
     }
    ],
    [
     {
      "node": "Discord: Normal Digest",
      "type": "main",
      "index": 0
     }
    ]
   ]
  },
  "Format Error": {
   "main": [
    [
     {
      "node": "Discord: Error Alert",
      "type": "main",
      "index": 0
     }
    ]
   ]
  }
 },
 "active": false,
 "settings": {
  "executionOrder": "v1"
 },
 "pinData": {},
 "meta": {
  "instanceId": "exported"
 }
}
```

Appendix B - Bonus_UptimeMonitor_YourName.json


```
{
 "name": "Bonus - Uptime Monitor",
 "nodes": [
  {
   "parameters": {
    "rule": {
     "interval": [
      {
       "field": "minutes",
       "minutesInterval": 5
      }
     ]
    }
   },
   "id": "d07a6e77-29a1-44d9-b1e6-e83f45338a37",
   "name": "Every 5 Minutes",
   "type": "n8n-nodes-base.scheduleTrigger",
   "typeVersion": 1.2,
   "position": [
    0,
    200
   ]
  },
  {
   "parameters": {
    "jsCode": "return [{ json: { t0: Date.now(), url: 'https://demo.realworld.io' } }];"
   },
   "id": "ec19b0cc-2f57-4c65-ba2f-d981b811ba26",
   "name": "Start Timer",
   "type": "n8n-nodes-base.code",
   "typeVersion": 2,
   "position": [
    220,
    200
   ]
  },
  {
   "parameters": {
    "url": "={{ $json.url }}",
    "options": {
     "timeout": 10000,
     "response": {
      "response": {
       "fullResponse": true,
       "neverError": true,
       "responseFormat": "text"
      }
     }
    }
   },
   "id": "e6a5d97e-e945-4c13-8220-15e437cfb828",
   "name": "HTTP Health Check",
   "type": "n8n-nodes-base.httpRequest",
   "typeVersion": 4.2,
   "position": [
    440,
    200
   ],
   "onError": "continueRegularOutput",
   "retryOnFail": true,
   "maxTries": 3,
   "waitBetweenTries": 3000
  },
  {
   "parameters": {
    "jsCode": "// Combine HTTP result + response time. Works for both success and error-output items.\nconst j = $input.first().json;\nconst t0 =
$('Start Timer').first().json.t0;\nconst ms = Date.now() - t0;\nconst code = Number(j.statusCode ?? (j.error && (j.error.httpCode || j.error.status))
?? 0) || 0;\nconst ok = code === 200;\n\n// keep rolling stats for the daily summary (persists only in active/production runs)\nconst s =
$getWorkflowStaticData('global');\ns.checks = (s.checks || 0) + 1;\ns.failures = (s.failures || 0) + (ok ? 0 : 1);\ns.totalMs = (s.totalMs || 0) +
ms;\ns.maxMs = Math.max(s.maxMs || 0, ms);\n\nconst reason = code === 0 ? 'no response / timeout / DNS error' : `HTTP ${code}`;\nreturn [{ json: { ok,
 code, ms,\n  alert: `■ **App DOWN**: ${$('Start Timer').first().json.url}\\nStatus: ${reason}\\nResponse time: ${ms} ms\\nTime: ${new
Date().toISOString()}` } }];"
   },
   "id": "c2c67e5e-04e5-40e6-a8c3-37bc280abc26",
   "name": "Evaluate Result",
   "type": "n8n-nodes-base.code",
   "typeVersion": 2,
   "position": [
    660,
    200
   ]
  },
  {
   "parameters": {
    "conditions": {
     "options": {
      "caseSensitive": true,
      "leftValue": "",
      "typeValidation": "strict"
     },
     "conditions": [
      {
       "id": "c1",
       "leftValue": "={{ $json.ok }}",
       "rightValue": "",
       "operator": {
        "type": "boolean",
        "operation": "false",
        "singleValue": true
       }
```

```
      }
     ],
     "combinator": "and"
    },
    "options": {}
   },
   "id": "b1faed7b-476d-4ac3-b973-03d33aba96f3",
   "name": "Is App Down?",
   "type": "n8n-nodes-base.if",
   "typeVersion": 2,
   "position": [
    880,
    200
   ]
  },
  {
   "parameters": {
    "authentication": "webhook",
    "content": "={{ $json.alert }}",
    "options": {}
   },
   "id": "015c5a88-e3f6-420a-a60b-7cb9a143b80f",
   "name": "Discord: Down Alert",
   "type": "n8n-nodes-base.discord",
   "typeVersion": 2,
   "position": [
    1120,
    200
   ],
   "credentials": {
    "discordWebhookApi": {
     "id": "REPLACE_DISCORD_CRED_ID",
     "name": "Discord Webhook (Morning Brief)"
    }
   },
   "onError": "continueRegularOutput"
  },
  {
   "parameters": {
    "rule": {
     "interval": [
      {
       "field": "days",
       "daysInterval": 1,
       "triggerAtHour": 9
      }
     ]
    }
   },
   "id": "f1c8aef9-0453-4964-b507-2b18e55884f7",
   "name": "Daily 9AM",
   "type": "n8n-nodes-base.scheduleTrigger",
   "typeVersion": 1.2,
   "position": [
    0,
    460
   ]
  },
  {
   "parameters": {
    "jsCode": "// Daily summary from stats collected by the 5-minute checks, then reset.\nconst s = $getWorkflowStaticData('global');\nconst checks =
s.checks || 0;\nconst fails = s.failures || 0;\nconst avg = checks ? Math.round(s.totalMs / checks) : 0;\nconst uptime = checks ? (((checks - fails) /
 checks) * 100).toFixed(2) : 'n/a';\nconst content = `■ **Daily Uptime Summary**\\nChecks: ${checks}\\nFailures: ${fails}\\nUptime: ${uptime}%\\nAvg
response: ${avg} ms\\nSlowest: ${s.maxMs || 0} ms`;\ns.checks = 0; s.failures = 0; s.totalMs = 0; s.maxMs = 0;\nreturn [{ json: { content } }];"
   },
   "id": "2212fd05-dc07-4534-abf2-1864b25264a1",
   "name": "Build Summary",
   "type": "n8n-nodes-base.code",
   "typeVersion": 2,
   "position": [
    440,
    460
   ]
  },
  {
   "parameters": {
    "authentication": "webhook",
    "content": "={{ $json.content }}",
    "options": {}
   },
   "id": "7ce3a75c-666a-4167-a3f1-aeb2687fcf4c",
   "name": "Discord: Daily Summary",
   "type": "n8n-nodes-base.discord",
   "typeVersion": 2,
   "position": [
    660,
    460
   ],
   "credentials": {
    "discordWebhookApi": {
     "id": "REPLACE_DISCORD_CRED_ID",
     "name": "Discord Webhook (Morning Brief)"
    }
   },
   "onError": "continueRegularOutput"
  }
 ],
 "connections": {
  "Every 5 Minutes": {
   "main": [
    [
     {
      "node": "Start Timer",
      "type": "main",
      "index": 0
```

```
    }
   ]
  ]
 },
 "Start Timer": {
  "main": [
   [
    {
     "node": "HTTP Health Check",
     "type": "main",
     "index": 0
    }
   ]
  ]
 },
 "HTTP Health Check": {
  "main": [
   [
    {
     "node": "Evaluate Result",
     "type": "main",
     "index": 0
    }
   ]
  ]
 },
 "Evaluate Result": {
  "main": [
   [
    {
     "node": "Is App Down?",
     "type": "main",
     "index": 0
    }
   ]
  ]
 },
 "Is App Down?": {
  "main": [
   [
    {
     "node": "Discord: Down Alert",
     "type": "main",
     "index": 0
    }
   ],
   []
  ]
 },
 "Daily 9AM": {
  "main": [
   [
    {
     "node": "Build Summary",
     "type": "main",
     "index": 0
    }
   ]
  ]
 },
 "Build Summary": {
  "main": [
   [
    {
     "node": "Discord: Daily Summary",
     "type": "main",
     "index": 0
    }
   ]
  ]
 }
},
"active": false,
"settings": {
 "executionOrder": "v1"
},
"pinData": {},
"meta": {
 "instanceId": "exported"
}
}
```